

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 15: AVALIAÇÃO DE ESTRATÉGIAS DE COMUNICAÇÃO MPI NA SIMULAÇÃO PARALELA DE DIFUSÃO DE CALOR

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 17/09/2026.

1. EXPERIMENTO

Implemente uma simulação da difusão de calor em uma barra 1D, dividida entre dois ou mais processos MPI. Cada processo deve simular um trecho da barra com células extras para troca de bordas com vizinhos. Implemente três versões: uma com MPI_Send/MPI_Recv, outra com MPI_Isend/MPI_Irecv e MPI_Wait, e uma terceira usando MPI_Test para atualizar os pontos internos enquanto aguarda a comunicação. Compare os tempos de execução e discuta os ganhos com sobreposição de comunicação e computação.

2. CONCEITOS RELEVANTES

- MPI_Isend e MPI_Irecv iniciam envios e recebimentos não bloqueantes.
  - **MPI_Isend:** começa o envio de dados.
  - **MPI_Irecv:** prepara o recebimento de dados.
- **MPI_Test** verifica se uma comunicação MPI não bloqueante já terminou, sem parar a execução do processo.

3. MÁQUINA UTILIZADA

Os experimentos foram executados na partição intel-128 do NPAD, composta por 55 nós Intel, totalizando 3.520 CPUs (média de 64 CPUs por nó) e 6.875 GB de memória, aproximadamente 125 GB por nó. A partição permite até 20 dias por job e configura 2GB de memória padrão por CPU (máximo de 4GB). Ela permite compartilhamento de nós, com a sobrescrição de até quatro jobs.

4. CÁLCULO DA BARRA 1D

Para simular a difusão de calor em uma barra 1D, ela é dividida em blocos contíguos, um para cada processo MPI. Cada processo armazena suas células locais e duas células extras, chamadas de halos ou ghost cells, que guardam as temperaturas recebidas dos processos vizinhos.

A cada passo de tempo, cada célula interna é atualizada pela equação de difusão:

$$T_i^{novo} = T_i + \alpha(T_{i-1} - 2T_i + T_{i+1})$$

Sendo:

- T_i^{novo} : nova temperatura da célula \(i\), no próximo passo da simulação.
- T_i: temperatura atual da célula \(i\).
- T_{i-1}: temperatura da célula imediatamente à esquerda.
- T_{i+1}: temperatura da célula imediatamente à direita.
- α: fator de difusão; controla quanto a temperatura pode mudar em cada passo. No código, é 0.24.
- $T_{i-1} - 2T_i + T_{i+1}$: diferença entre a temperatura da célula e a influência dos dois vizinhos; representa a curvatura térmica local.

Como as células nas bordas de um bloco dependem de valores que pertencem aos vizinhos, os processos trocam seus halos antes de calcular essas bordas. As extremidades globais da barra mantêm temperaturas fixas, neste caso 100 e 0 graus.

Depois de calcular as novas temperaturas, os vetores atual e próximo são trocados, e o processo se repete pelo número definido de passos.

5. CÓDIGO

A seguir, seguem três versões de código que simulam a mesma difusão de calor e usam a mesma divisão da barra entre processos MPI. A diferença está apenas em como trocam as temperaturas das células de fronteira entre processos, sendo elas:

- **calorBloqueante**: usa MPI_Send e MPI_Recv. Cada processo envia e recebe os halos de modo bloqueante, esperando a comunicação terminar antes de calcular todas as suas células.
- **calorNaoBloqueante**: usa MPI_Isend e MPI_Irecv para iniciar os envios e recebimentos sem bloquear imediatamente. Porém, logo depois usa MPI_Wait; assim, ainda espera toda a comunicação terminar antes de iniciar o cálculo. Evita uma ordem rígida de comunicação, mas não sobrepõe comunicação com cálculo.
- **calorComSobreposicao**: também inicia a comunicação com MPI_Isend e MPI_Irecv, mas enquanto os halos são recebidos calcula primeiro as células internas, que não dependem dos vizinhos. Periodicamente usa MPI_Test para verificar se os halos chegaram; ao fim, calcula as células das bordas. Assim, tenta executar comunicação e cálculo ao mesmo tempo.

Figura 1: calorBloqueante

```c
#include "calorComum.h"

static void trocarHalosBloqueante(
    SimulacaoCalor *simulacao,
    int identificadorProcesso,
    int quantidadeProcessos,
    MPI_Comm comunicador
) {
    int vizinhoEsquerda = identificadorProcesso - 1;
    int vizinhoDireita = identificadorProcesso + 1;
    int ultimaCelula = simulacao->quantidadeCelulasLocais;

    /* Pares e impares seguem ordens opostas para evitar espera circular. */
    // Se o processo for par, ele envia primeiro para a direita e depois para a esquerda.
    if (identificadorProcesso % 2 == 0) {
```

```
        // verifica se o vizinho da direita existe antes de enviar e receber
        if (vizinhoDireita < quantidadeProcessos)
            MPI_Send(&simulacao->temperaturaAtual[ultimaCelula], 1, MPI_DOUBLE,
                    vizinhoDireita, tagParaEsquerda, comunicador);
        // verifica se o vizinho da direita existe antes de enviar e receber
        if (vizinhoDireita < quantidadeProcessos)
            MPI_Recv(&simulacao->temperaturaAtual[ultimaCelula + 1], 1, MPI_DOUBLE,
                    vizinhoDireita, tagParaDireita, comunicador, MPI_STATUS_IGNORE);
        // verifica se o vizinho da esquerda existe antes de enviar e receber
        if (vizinhoEsquerda >= 0)
            MPI_Send(&simulacao->temperaturaAtual[1], 1, MPI_DOUBLE,
                    vizinhoEsquerda, tagParaDireita, comunicador);
        if (vizinhoEsquerda >= 0)
            MPI_Recv(&simulacao->temperaturaAtual[0], 1, MPI_DOUBLE,
                    vizinhoEsquerda, tagParaEsquerda, comunicador, MPI_STATUS_IGNORE);
    // Se o processo for ímpar, ele recebe primeiro da esquerda e depois da direita
    } else {
        // verifica se o vizinho da esquerda existe antes de enviar e receber
        if (vizinhoEsquerda >= 0)
            MPI_Recv(&simulacao->temperaturaAtual[0], 1, MPI_DOUBLE,
                    vizinhoEsquerda, tagParaEsquerda, comunicador, MPI_STATUS_IGNORE);
        if (vizinhoEsquerda >= 0)
            MPI_Send(&simulacao->temperaturaAtual[1], 1, MPI_DOUBLE,
                    vizinhoEsquerda, tagParaDireita, comunicador);
        // verifica se o vizinho da direita existe antes de enviar e receber
        if (vizinhoDireita < quantidadeProcessos)
            MPI_Recv(&simulacao->temperaturaAtual[ultimaCelula + 1], 1, MPI_DOUBLE,
                    vizinhoDireita, tagParaDireita, comunicador, MPI_STATUS_IGNORE);
        if (vizinhoDireita < quantidadeProcessos)
            MPI_Send(&simulacao->temperaturaAtual[ultimaCelula], 1, MPI_DOUBLE,
                    vizinhoDireita, tagParaEsquerda, comunicador);
    }
}



static void executarPassoBloqueante(
    SimulacaoCalor *simulacao,
    int identificadorProcesso,
    int quantidadeProcessos,
    MPI_Comm comunicador
```

```
) {
    int inicioAtualizacao;
    int fimAtualizacao;

    trocarHalosBloqueante(simulacao, identificadorProcesso, quantidadeProcessos,
comunicador);
    prepararProximoPasso(simulacao, &inicioAtualizacao, &fimAtualizacao);
    calcularIntervalo(simulacao, inicioAtualizacao, fimAtualizacao);
    trocarTemperaturas(simulacao);
}

int main(int quantidadeArgumentos, char **argumentos) {
    MPI_Init(&quantidadeArgumentos, &argumentos);

    int identificadorProcesso, quantidadeProcessos, quantidadeCelulas, quantidadePassos;
    double fatorDifusao;

    MPI_Comm_rank(MPI_COMM_WORLD, &identificadorProcesso);
    MPI_Comm_size(MPI_COMM_WORLD, &quantidadeProcessos);

    lerArgumentos(
        quantidadeArgumentos,
        argumentos,
        &quantidadeCelulas,
        &quantidadePassos,
        &fatorDifusao,
        MPI_COMM_WORLD
    );

    SimulacaoCalor simulacao;

    inicializarSimulacao(&simulacao, quantidadeCelulas, fatorDifusao, MPI_COMM_WORLD);
    MPI_Barrier(MPI_COMM_WORLD);

    double inicio = MPI_Wtime();
    for (int passo = 0; passo < quantidadePassos; ++passo){
        executarPassoBloqueante(
            &simulacao,
            identificadorProcesso,
            quantidadeProcessos,
            MPI_COMM_WORLD
```

```
        );
    }
    mostrarTempo(&simulacao, MPI_Wtime() - inicio, "MPI_Send/MPI_Recv",
MPI_COMM_WORLD);
    liberarSimulacao(&simulacao);
    MPI_Finalize();

    return 0;
}
```

Figura 2: calorNaoBloqueante

```
#include "calorComum.h"

static void executarPassoComEspera(
    SimulacaoCalor *simulacao,
    int identificadorProcesso,
    int quantidadeProcessos,
    MPI_Comm comunicador
) {
    MPI_Request requisicoes[4];
    int quantidadeRecebimentos, inicioAtualizacao, fimAtualizacao;

    int quantidadeRequisicoes = iniciarTrocasNaoBloqueantes(
        simulacao,
        identificadorProcesso,
        quantidadeProcessos,
        requisicoes,
        &quantidadeRecebimentos,
        comunicador
    );

    (void)quantidadeRecebimentos;

    // Espera até que todas as requisições sejam concluídas.
    for (int indice = 0; indice < quantidadeRequisicoes; ++indice) {
        MPI_Wait(&requisicoes[indice], MPI_STATUS_IGNORE);
    }
    prepararProximoPasso(simulacao, &inicioAtualizacao, &fimAtualizacao);
    calcularIntervalo(simulacao, inicioAtualizacao, fimAtualizacao);
    trocarTemperaturas(simulacao);
```

```
}

int main(int quantidadeArgumentos, char **argumentos) {
  MPI_Init(&quantidadeArgumentos, &argumentos);

  int identificadorProcesso, quantidadeProcessos, quantidadeCelulas, quantidadePassos;
  double fatorDifusao;

  MPI_Comm_rank(MPI_COMM_WORLD, &identificadorProcesso);
  MPI_Comm_size(MPI_COMM_WORLD, &quantidadeProcessos);

  lerArgumentos(
    quantidadeArgumentos,
    argumentos,
    &quantidadeCelulas,
    &quantidadePassos,
    &fatorDifusao,
    MPI_COMM_WORLD
  );

  SimulacaoCalor simulacao;

  inicializarSimulacao(&simulacao, quantidadeCelulas, fatorDifusao, MPI_COMM_WORLD);

  MPI_Barrier(MPI_COMM_WORLD);

  double inicio = MPI_Wtime();
  // Executa a simulação com trocas não bloqueantes e espera ativa.
  for (int passo = 0; passo < quantidadePassos; ++passo) {
    executarPassoComEspera(&simulacao, identificadorProcesso, quantidadeProcessos,
MPI_COMM_WORLD);
  }
  mostrarTempo(&simulacao, MPI_Wtime() - inicio, "MPI_Isend/Irecv + Wait",
MPI_COMM_WORLD);

  liberarSimulacao(&simulacao);
  MPI_Finalize();

  return 0;
}
```

Figura 3: calorComSobreposicao

```c
#include "calorComum.h"

#define INTERVALO_MPI_TEST 100

#define TEXTO_INTERNO(valor) #valor
#define TEXTO(valor) TEXTO_INTERNO(valor)

static int recebimentosConcluidos(MPI_Request requisicoes[4], int
quantidadeRecebimentos) {
    int todosConcluidos = 1;

    for (int indice = 0; indice < quantidadeRecebimentos; ++indice) {
        int concluiu;
        // Testa se a requisição foi concluída sem bloquear a execução.
        MPI_Test(&requisicoes[indice], &concluiu, MPI_STATUS_IGNORE);
        // Se alguma requisição não foi concluída, marca como falso.
        todosConcluidos &= concluiu;
    }

    return todosConcluidos;
}

static void executarPassoComSobreposicao(
    SimulacaoCalor *simulacao,
    int identificadorProcesso,
    int quantidadeProcessos,
    MPI_Comm comunicador
) {
    MPI_Request requisicoes[4];

    int quantidadeRecebimentos, inicioAtualizacao, fimAtualizacao;
    int quantidadeRequisicoes = iniciarTrocasNaoBloqueantes(
        simulacao,
        identificadorProcesso,
        quantidadeProcessos,
        requisicoes,
        &quantidadeRecebimentos,
        comunicador
    );
```

```
  prepararProximoPasso(simulacao, &inicioAtualizacao, &fimAtualizacao);

  int inicioInterno = inicioAtualizacao > 2 ? inicioAtualizacao : 2;

  int fimInterno;
  // Se o fim da atualização for menor que a última célula local,
  // define o fim interno como o fim da atualização.
  // Caso contrário, define como a penúltima célula local.
  if (fimAtualizacao < simulacao->quantidadeCelulasLocais - 1){
     fimInterno = fimAtualizacao;
  } else {
     fimInterno = simulacao->quantidadeCelulasLocais - 1;
  }

  int indiceLocal = inicioInterno;
  int halosRecebidos = 0;
  // Enquanto não tiver recebido os halos,
  // calcula blocos de células e verifica se os halos foram recebidos.
  while (indiceLocal <= fimInterno && !halosRecebidos) {
     int fimBloco = indiceLocal + INTERVALO_MPI_TEST - 1;
     if (fimBloco > fimInterno) {
        fimBloco = fimInterno;
     }

     calcularIntervalo(simulacao, indiceLocal, fimBloco);
     indiceLocal = fimBloco + 1;
     halosRecebidos = recebimentosConcluidos(requisicoes, quantidadeRecebimentos);
  }

  calcularIntervalo(simulacao, indiceLocal, fimInterno);

      // Se ainda não recebeu os halos, espera até que todas as requisições sejam
concluídas.
  if (!halosRecebidos) {
     for (int indice = 0; indice < quantidadeRequisicoes; ++indice) {
        MPI_Wait(&requisicoes[indice], MPI_STATUS_IGNORE);
     }
  }

  // Calcula as células das bordas, se necessário.
```

```
    // Calcula a primeira célula local se estiver dentro do intervalo de atualização.
    if (inicioAtualizacao <= 1 && fimAtualizacao >= 1){
        calcularCelula(simulacao, 1);
    }
    // Calcula a última célula local se estiver dentro do intervalo de atualização.
    if (inicioAtualizacao <= simulacao->quantidadeCelulasLocais &&
        fimAtualizacao >= simulacao->quantidadeCelulasLocais) {
            calcularCelula(simulacao, simulacao->quantidadeCelulasLocais);
        }
    // Se ainda não recebeu os halos, espera até que todas as requisições sejam concluídas.
    if (halosRecebidos) {
        for (int indice = 0; indice < quantidadeRequisicoes; ++indice) {
            MPI_Wait(&requisicoes[indice], MPI_STATUS_IGNORE);
        }
    }

    // Troca os ponteiros das temperaturas atual e próxima.
    trocarTemperaturas(simulacao);

}

int main(int quantidadeArgumentos, char **argumentos) {
    MPI_Init(&quantidadeArgumentos, &argumentos);

    int identificadorProcesso, quantidadeProcessos, quantidadeCelulas, quantidadePassos;
    double fatorDifusao;

    MPI_Comm_rank(MPI_COMM_WORLD, &identificadorProcesso);
    MPI_Comm_size(MPI_COMM_WORLD, &quantidadeProcessos);

    lerArgumentos(
        quantidadeArgumentos,
        argumentos,
        &quantidadeCelulas,
        &quantidadePassos,
        &fatorDifusao,
        MPI_COMM_WORLD
    );

    SimulacaoCalor simulacao;
```

```
  inicializarSimulacao(
    &simulacao,
    quantidadeCelulas,
    fatorDifusao,
    MPI_COMM_WORLD
  );

  MPI_Barrier(MPI_COMM_WORLD);
  double inicio = MPI_Wtime();

  for (int passo = 0; passo < quantidadePassos; ++passo){
    executarPassoComSobreposicao(
      &simulacao,
      identificadorProcesso,
      quantidadeProcessos,
      MPI_COMM_WORLD
    );
  }

  mostrarTempo(
    &simulacao,
    inicio,
    "MPI_Test a cada " TEXTO(INTERVALO_MPI_TEST),
    MPI_COMM_WORLD
  );
  liberarSimulacao(&simulacao);
  MPI_Finalize();

  return 0;
}
```

6. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Tabela 1: Mediana de 7 execuções dos dados com 2 processos e 2 nós

Células	Bloqueante (s)	Não bloqueante (s)	Sobreposição (s)

100.000	0,033943	0,028408	0,028026
1.000.000	0,266118	0,246401	0,241261
2.000.000	0,463515	0,45306	0,455261
5.000.000	1,135849	1,164911	1,189348
7.000.000	2,459573	2,458808	2,469876
10.000.000	3,630785	3,63586	3,693392

Tabela 2: Mediana de 7 execuções dos dados com 8 processos e 8 nós

Células	Bloqueante (s)	Não bloqueante (s)	Sobreposição (s)
100.000	0,039266	0,018316	0,018187
1.000.000	0,091013	0,075661	0,066392
2.000.000	0,204244	0,149623	0,145259
5.000.000	0,765191	0,329194	0,318169
7.000.000	0,463519	0,450820	0,417993
10.000.000	0,595209	0,575042	0,563194

Figura 4: Mediana de 7 execuções dos dados com 2 processos e 2 nós

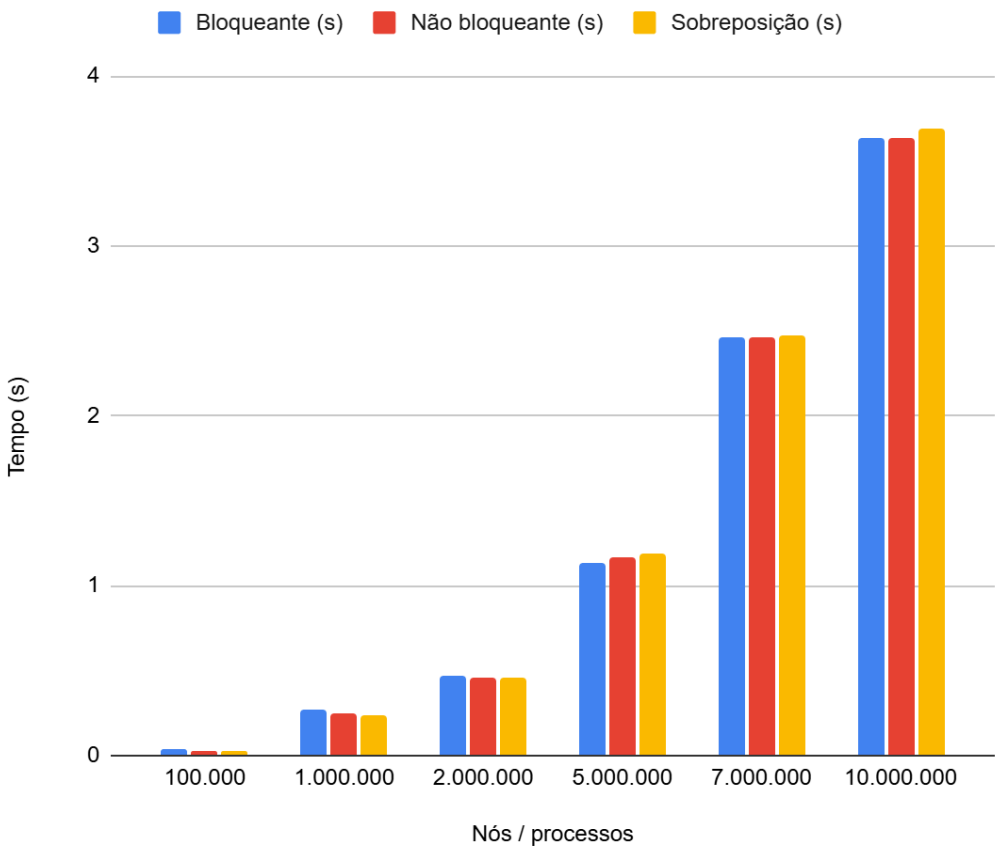
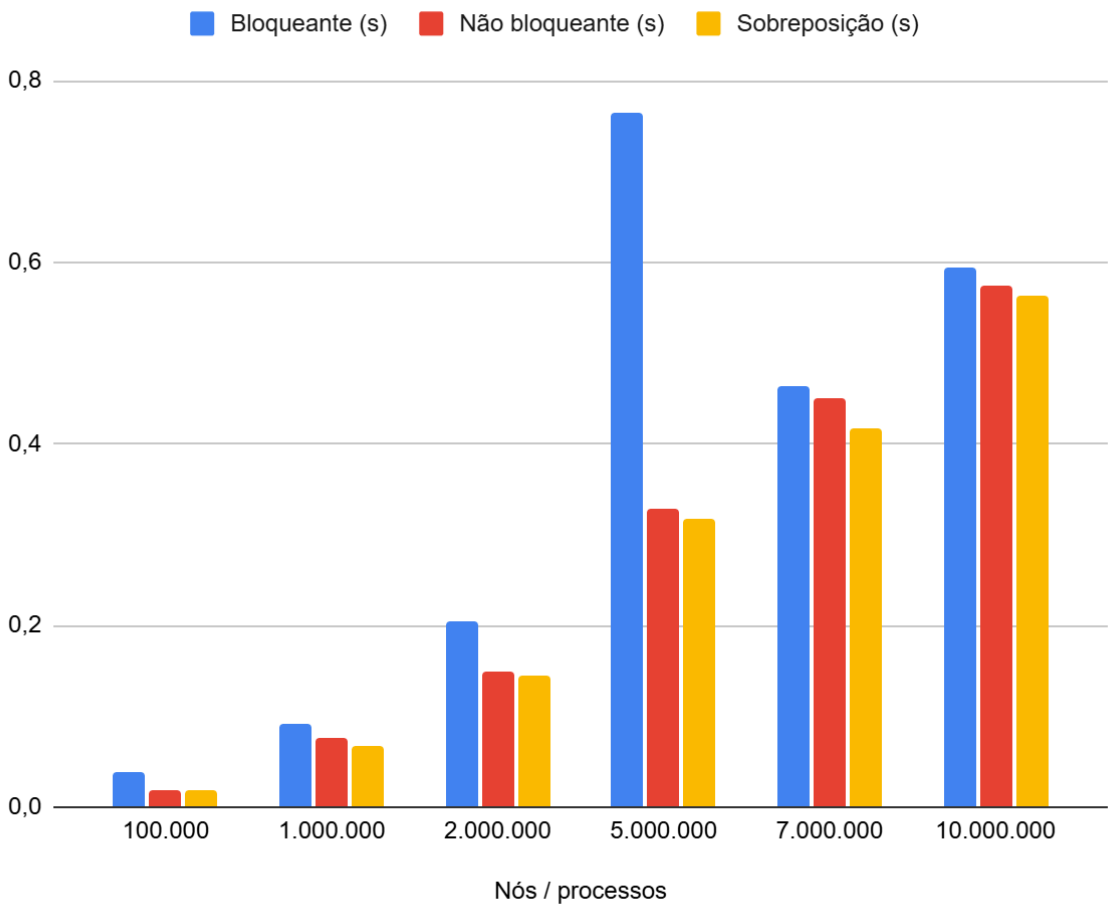


Figura 5: Mediana de 7 execuções dos dados com 8 processos e 8 nós



Os resultados mostram que o uso de comunicação não bloqueante tende a reduzir o tempo de execução, principalmente com 8 processos. Isso ocorre porque a versão bloqueante usa “MPI_Send” e “MPI_Recv” em sequência: antes de calcular o próximo passo da simulação, cada processo precisa concluir a troca dos valores de fronteira com seus vizinhos. Portanto, parte do tempo de processamento fica ociosa, aguardando comunicação.

Na versão não bloqueante, os recebimentos e envios são publicados antecipadamente com “MPI_Irecv” e “MPI_Isend”. Embora o programa ainda use “MPI_Wait” antes do cálculo, essa organização reduz restrições de sincronização entre processos e pode permitir que a biblioteca MPI progrida a comunicação mais eficientemente. Por isso, essa versão foi frequentemente mais rápida que a bloqueante, especialmente nos testes com 8 processos.

A versão com sobreposição tende a apresentar os melhores resultados porque, após iniciar a comunicação, ela calcula as células internas do subdomínio.

Essas células não dependem dos halos recebidos dos processos vizinhos. Somente as células localizadas nas extremidades precisam esperar os dados remotos. Assim, o programa tenta usar o tempo de comunicação para realizar cálculo útil, verificando periodicamente a chegada dos halos com “MPI_Test”.

Com 2 processos, o benefício dessa técnica é limitado. Há apenas uma fronteira entre os dois subdomínios e cada processo possui uma quantidade grande de células para calcular. Como a troca envia somente um valor “double” por direção, o custo de comunicação representa uma parcela pequena do tempo total. Nesse cenário, o custo adicional de chamadas a “MPI_Test” pode anular o benefício da sobreposição, explicando por que os tempos ficam muito próximos, e por que, em domínios maiores, a versão bloqueante pode até ser ligeiramente melhor.

Com 8 processos, cada processo recebe uma parcela menor do domínio e existem mais processos que precisam se coordenar. Dessa forma, a latência e a sincronização da comunicação passam a ter maior impacto relativo. Por isso, as versões não bloqueantes tiveram ganhos mais claros, chegando a aproximadamente 54% para 100 mil células e cerca de 27% a 29% para 1 e 2 milhões de células.

7. ANEXO DE CÓDIGO DE DIFUSÃO DA BARRA

Figura 4: calorComum.h

```h
#ifndef CALOR_COMUM_H
#define CALOR_COMUM_H

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

enum { tagParaEsquerda = 10, tagParaDireita = 11 };

typedef struct {
    int quantidadeCelulas;
    int quantidadeCelulasLocais;
    int primeiraCelulaGlobal;
    double fatorDifusao;
    double temperaturaEsquerda;
    double temperaturaDireita;
```

```c
    double *temperaturaAtual;
    double *proximaTemperatura;
} SimulacaoCalor;

static inline void encerrarComErro(int condicao, const char *mensagem, MPI_Comm
comunicador) {
    if (condicao) {
        fprintf(stderr, "%s\n", mensagem);
        MPI_Abort(comunicador, 1);
    }
}

static inline void lerArgumentos(
    int quantidadeArgumentos,
    char **argumentos,
    int *quantidadeCelulas,
    int *quantidadePassos,
    double *fatorDifusao,
    MPI_Comm comunicador
) {
    *quantidadeCelulas = quantidadeArgumentos > 1 ? atoi(argumentos[1]) : 1000000;
    *quantidadePassos = quantidadeArgumentos > 2 ? atoi(argumentos[2]) : 1000;
    *fatorDifusao = quantidadeArgumentos > 3 ? atof(argumentos[3]) : 0.24;

    encerrarComErro(
        *quantidadeCelulas < 4 || *quantidadePassos < 1 || *fatorDifusao <= 0.0 || *fatorDifusao >
0.5,
        "Uso: programa [celulas>=4] [passos>=1] [alpha em (0,0.5)]",
        comunicador
    );
}

static inline void inicializarSimulacao(SimulacaoCalor *simulacao, int quantidadeCelulas,
double fatorDifusao, MPI_Comm comunicador) {
    int identificadorProcesso, quantidadeProcessos;
    MPI_Comm_rank(comunicador, &identificadorProcesso); MPI_Comm_size(comunicador,
&quantidadeProcessos);
    encerrarComErro(quantidadeProcessos < 2, "Execute com pelo menos 2 processos MPI.",
comunicador);
    encerrarComErro(quantidadeCelulas < 2 * quantidadeProcessos, "N deve ser pelo menos
duas vezes o numero de processos.", comunicador);
```

```
    int tamanhoBase = quantidadeCelulas / quantidadeProcessos;
    int celulasRestantes = quantidadeCelulas % quantidadeProcessos;
    simulacao->quantidadeCelulas = quantidadeCelulas;
    simulacao->quantidadeCelulasLocais = tamanhoBase + (identificadorProcesso <
celulasRestantes);
    simulacao->primeiraCelulaGlobal = identificadorProcesso * tamanhoBase +
(identificadorProcesso < celulasRestantes ? identificadorProcesso : celulasRestantes);
    simulacao->fatorDifusao = fatorDifusao; simulacao->temperaturaEsquerda = 100.0;
simulacao->temperaturaDireita = 0.0;
    simulacao->temperaturaAtual = calloc((size_t)simulacao->quantidadeCelulasLocais + 2,
sizeof *simulacao->temperaturaAtual);
    simulacao->proximaTemperatura = calloc((size_t)simulacao->quantidadeCelulasLocais + 2,
sizeof *simulacao->proximaTemperatura);
    encerrarComErro(!simulacao->temperaturaAtual || !simulacao->proximaTemperatura,
"Falha ao alocar memoria.", comunicador);
    for (int indiceLocal = 1; indiceLocal <= simulacao->quantidadeCelulasLocais;
++indiceLocal) {
        int indiceGlobal = simulacao->primeiraCelulaGlobal + indiceLocal - 1;
        simulacao->temperaturaAtual[indiceLocal] = 20.0 + 10.0 * indiceGlobal /
(double)(quantidadeCelulas - 1);
    }
    if (simulacao->primeiraCelulaGlobal == 0) simulacao->temperaturaAtual[1] =
simulacao->temperaturaEsquerda;
    if (simulacao->primeiraCelulaGlobal + simulacao->quantidadeCelulasLocais ==
quantidadeCelulas) simulacao->temperaturaAtual[simulacao->quantidadeCelulasLocais] =
simulacao->temperaturaDireita;
}

static inline void prepararProximoPasso(SimulacaoCalor *simulacao, int *inicioAtualizacao,
int *fimAtualizacao) {
    *inicioAtualizacao = 1; *fimAtualizacao = simulacao->quantidadeCelulasLocais;
    if (simulacao->primeiraCelulaGlobal == 0) { simulacao->proximaTemperatura[1] =
simulacao->temperaturaEsquerda; *inicioAtualizacao = 2; }
    if (simulacao->primeiraCelulaGlobal + simulacao->quantidadeCelulasLocais ==
simulacao->quantidadeCelulas) {
simulacao->proximaTemperatura[simulacao->quantidadeCelulasLocais] =
simulacao->temperaturaDireita; *fimAtualizacao = simulacao->quantidadeCelulasLocais - 1; }
}

static inline void calcularCelula(SimulacaoCalor *simulacao, int indiceLocal) {
    double *temperaturas = simulacao->temperaturaAtual;
```

```
    simulacao->proximaTemperatura[indiceLocal] = temperaturas[indiceLocal] +
simulacao->fatorDifusao * (temperaturas[indiceLocal - 1] - 2.0 * temperaturas[indiceLocal] +
temperaturas[indiceLocal + 1]);
}
static inline void calcularIntervalo(SimulacaoCalor *simulacao, int inicio, int fim) { for (int
indiceLocal = inicio; indiceLocal <= fim; ++indiceLocal) calcularCelula(simulacao,
indiceLocal); }
static inline void trocarTemperaturas(SimulacaoCalor *simulacao) { double *temporario =
simulacao->temperaturaAtual; simulacao->temperaturaAtual =
simulacao->proximaTemperatura; simulacao->proximaTemperatura = temporario; }

static inline int iniciarTrocasNaoBloqueantes(
    SimulacaoCalor *simulacao,
    int identificadorProcesso,
    int quantidadeProcessos,
    MPI_Request requisicoes[4],
    int *quantidadeRecebimentos,
    MPI_Comm comunicador
) {
    int quantidadeRequisicoes = 0;

    // Se o processo não for o primeiro, ele deve receber a temperatura do processo à
esquerda.
    if (identificadorProcesso > 0){
        MPI_Irecv(
            &simulacao->temperaturaAtual[0],
            1,
            MPI_DOUBLE,
            identificadorProcesso - 1,
            tagParaEsquerda,
            comunicador,
            &requisicoes[quantidadeRequisicoes++]
        );
    }

    // Se o processo não for o último, ele deve receber a temperatura do processo à direita.
    if (identificadorProcesso < quantidadeProcessos - 1){
        MPI_Irecv(
            &simulacao->temperaturaAtual[simulacao->quantidadeCelulasLocais + 1],
            1,
            MPI_DOUBLE,
```

```
            identificadorProcesso + 1,
            tagParaDireita,
            comunicador,
            &requisicoes[quantidadeRequisicoes++]
        );
    }

    // Se o processo não for o primeiro, ele deve enviar a temperatura da primeira célula local para o processo à esquerda.
    *quantidadeRecebimentos = quantidadeRequisicoes;
    if (identificadorProcesso > 0) {
        MPI_Isend(
            &simulacao->temperaturaAtual[1],
            1,
            MPI_DOUBLE,
            identificadorProcesso - 1,
            tagParaDireita,
            comunicador,
            &requisicoes[quantidadeRequisicoes++]
        );
    }
    // Se o processo não for o último, ele deve enviar a temperatura da última célula local para o processo à direita.
    if (identificadorProcesso < quantidadeProcessos - 1) {
        MPI_Isend(
            &simulacao->temperaturaAtual[simulacao->quantidadeCelulasLocais],
            1,
            MPI_DOUBLE,
            identificadorProcesso + 1,
            tagParaEsquerda,
            comunicador,
            &requisicoes[quantidadeRequisicoes++]
        );
    }

    return quantidadeRequisicoes;
}

static inline void mostrarTempo(const SimulacaoCalor *simulacao, double inicio, const char *nomeVersao, MPI_Comm comunicador) {
    int identificadorProcesso;
```

```
    MPI_Comm_rank(comunicador, &identificadorProcesso);
    MPI_Barrier(comunicador);
    if (identificadorProcesso == 0) printf("%-22s tempo_total = %.6f s | T[0] = %.6f\n",
nomeVersao, MPI_Wtime() - inicio, simulacao->temperaturaAtual[1]);
}
static inline void liberarSimulacao(SimulacaoCalor *simulacao) {
free(simulacao->temperaturaAtual); free(simulacao->proximaTemperatura); }

#endif
```